



Universidad Central de Venezuela
Facultad de Ciencias
Escuela de Computación

Gestión de contenidos sobre arquitecturas web de última generación

Seminario presentado ante la ilustre
Universidad Central de Venezuela por:
Br. Juan Luis Sánchez Téllez

Tutor: Prof. Walter Hernández

Caracas, Agosto de 2015

Resumen

En el siguiente trabajo se describen un conjunto de términos y tecnologías relacionadas con el desarrollo de portales web orientados a la gestión de usuarios y documentos. En la actualidad, existen diversas tecnologías y arquitecturas que sirven para desarrollar este tipo de aplicaciones web. Entre las tecnologías más actuales se encuentra la plataforma asíncrona *Node.js*, que permite sacar provecho de la velocidad y del ambiente no bloqueante que posee *Javascript* como lenguaje de programación. También existen arquitecturas de desarrollo separadas en contenedores del lado del servidor y del lado del cliente, como es el caso de un *Web API* y una aplicación de una sola página o *SPA* (*Single-page application* por sus siglas en inglés). Después de recopilar diversas maneras de desarrollar un portal web interactivo, se escogerán las más novedosas con sus debidas justificaciones, para así dar una propuesta a una nueva versión del portal generador de sitios web de la Facultad de Ciencias de la Universidad Central de Venezuela que permita solventar algunas limitaciones de las versiones actualmente en producción (*Genasig* y *Portalsig*).

Palabras claves: Web 2.0, Portal, Web API, SPA, MVC, ORM

Tabla de Contenidos

Introducción	VI
1. Web 2.0	1
1.1. Principios	2
1.2. Aplicación web	3
2. Portal web	5
2.1. Clasificación	5
2.1.1. Horizontal	5
2.1.2. Vertical	6
2.2. Elementos	7
3. Tendencias tecnológicas	10
3.1. Arquitecturas y patrones	10
3.1.1. MVC	10
3.1.2. ORM	12
3.1.3. Web APIs y SPA	13
3.2. Lenguajes y Frameworks	15
3.2.1. Ruby	16
3.2.2. Python	17
3.2.3. Javascript	18
3.3. Base de datos	20
3.4. Pilas de desarrollo web	20
3.4.1. LAMP	20
3.4.2. MEAN	21
4. Portalsig	23
4.1. Funcionalidades generales	24
4.2. Estado actual	25
4.3. Tecnologías utilizadas	26
5. Propuesta de trabajo especial de grado	27
5.1. Planteamiento y justificación	27

5.2. Objetivo general	27
5.3. Objetivos específicos	27
5.4. Metodología	28
5.5. Tecnologías	28
5.6. Innovaciones propuestas	29

Lista de Figuras

1.1. Interacción de los diferentes principios de la <i>Web 2.0</i>	3
2.1. Diferencia entre portales <i>horizontales</i> y <i>verticales</i> en base a los usuarios que los utilizan.	6
3.1. Colaboración de los componentes del patrón <i>MVC</i>	12
3.2. Integración de la técnica <i>ORM</i> con el <i>Modelo</i> del patrón <i>MVC</i>	13
3.3. Arquitectura de una aplicación web que consta de un <i>Web API</i> y una <i>SPA</i> . . .	15
3.4. Pila de desarrollo web <i>LAMP</i>	21
3.5. Pila de desarrollo web <i>MEAN</i>	22
4.1. Arquitectura actual de <i>Portalsig</i>	26
5.1. Arquitectura propuesta para la nueva versión de <i>Portalsig</i>	29

Introducción

En los últimos años con el auge de Internet y el rápido acceso a la información que le proporciona a cualquier usuario, la mayoría de las empresas, instituciones, universidades, comercios y otros entes, lo han tomado como una herramienta para facilitar diversos procesos que antes eran complicados y tediosos, agregado a esto, actualmente, nos encontramos en una época donde las aplicaciones móviles y las aplicaciones web interactivas son el gran común en soluciones de software y las predilectas por la mayoría de los usuarios.

Es por esto que la Escuela de Computación de la Facultad de Ciencias de la Universidad Central de Venezuela ha decidido crear un nuevo portal generador de asignaturas, para el uso exclusivo de la Facultad de Ciencias, para de este modo, permitirle tanto a los docentes como a los estudiantes la gestión de manera eficaz y eficiente de sus asignaturas correspondientes.

La finalidad principal de este futuro TEG es automatizar y facilitar diversos procesos referentes a la gestión de asignaturas de una forma elegante empleando las últimas tecnologías disponibles, contribuyendo con todos los estudiantes y docentes de la Facultad de Ciencias.

Capítulo 1

Web 2.0

El término *Web 2.0* comprende aquellos sitios web que facilitan el compartir información, la interoperabilidad, el diseño centrado en el usuario y la colaboración en la *World Wide Web*. Un sitio *Web 2.0* permite a los usuarios interactuar y colaborar entre sí como creadores de contenido generado por usuarios en una comunidad virtual. Ejemplos de la *Web 2.0* son las comunidades web, los servicios web, las aplicaciones web, los servicios de red social, los servicios de alojamiento de videos, las wikis, blogs, mashups y folcsonomías.

La *Web 2.0* no es más que la evolución de la Web o Internet en el que los usuarios dejan de ser usuarios pasivos para convertirse en usuarios activos, que participan y contribuyen en el contenido de la red siendo capaces de dar soporte y formar parte de una sociedad que se informa, comunica y genera conocimiento. La *Web 2.0* es un concepto que se introdujo en 2003 y que se refiere al fenómeno social surgido a partir del desarrollo de diversas aplicaciones en Internet. El término establece una distinción entre la primera época de la *Web* (donde el usuario era básicamente un sujeto pasivo que recibía la información o la publicaba, sin que existieran demasiadas posibilidades para que se generara la interacción) y la revolución que supuso el auge de los blogs, las redes sociales y otras herramientas relacionadas.

Antes de la llegada de las tecnologías de la *Web 2.0* se utilizaban páginas estáticas programadas en *HTML* que no eran actualizadas frecuentemente. El éxito del mundo web dependía de páginas webs más dinámicas (a veces llamadas *Web 1.5*) donde los sistemas de gestión de contenidos servían páginas *HTML* dinámicas creadas al instante desde una base de datos actualizada. En ambos sentidos, el conseguir *hits* (visitas) y la estética visual eran considerados como factores.

Los teóricos de la aproximación a la *Web 2.0* (Tim O'Reilly) piensan que el uso de la misma está orientado a la interacción y redes sociales, que pueden servir contenido que explota los efectos de las redes, creando o no sitios webs interactivas y visuales. Es decir, los sitios *Web 2.0* actúan más como puntos de encuentro o páginas webs dependientes de usuarios, que como páginas tradicionales.

1.1. Principios

En el 2005, Tim O'Reilly identificó algunas diferencias entre la *Web 1.0* con respecto a la *Web 2.0* y precisó siete (7) principios que describen los cambios más notables los cuales son:

- Utilización de la World Wide Web como plataforma, debido a que la *Web 2.0* se utiliza para mantener todas las aplicaciones y compartirlas. Muchas de éstas de forma gratuita, por lo que, la información de un usuario es guardada en la red y se puede acceder a ésta en cualquier momento a través de Internet.
- Aprovechamiento de la inteligencia colectiva, ya que mientras más usuarios utilicen aplicaciones *Web 2.0*, más usuarios crearán y compartirán nuevos contenidos. Esto convierte a los usuarios tanto en productores como en consumidores del contenido web.
- La gestión de bases de datos como competencia básica, debido a que el valor de las aplicaciones está determinado por los datos que guarda y que hace disponible al público, estos datos deben ser íntegros y consistentes en todo momento.
- Construcción de aplicaciones incrementales, lo que permite ampliar el conjunto de funcionalidades de un sistema (*Escalabilidad*).
- Uso de modelos de programación ligeros, ya que las herramientas para crear aplicaciones *Web 2.0* son simples, al igual que la interacción entre éstas.
- Expansión de las plataformas que soportan el software, para que pueda ser utilizado desde computadores, teléfonos inteligentes, consolas de juegos o reproductores de música, entre otros.
- Actualización constante de las aplicaciones, manteniendo a los usuarios que las utilizan entretenidos y atraídos hacia ésta.

En la *Figura 1.1* se puede observar como los *principios* anteriormente descritos, pueden ser visto como componentes dentro de la *Web 2.0* como arquitectura. Puede ser dividido en cuatro (4) grandes bloques: *usuarios*, *clientes web*, *servicios web* y *datos*. Los *usuarios* son los protagonistas en el término de *Web 2.0*, ya que son los que usan las aplicaciones y participan en como la información puede ser agregada y modificada, con la posibilidad de formar comunidades dando vida a la *inteligencia colectiva*. Esta interacción de los *usuarios* con el resto de la *Web 2.0* es posible a través de clientes web en diferentes *plataformas*, que pueden ser navegadores, aplicaciones móviles, consolas de videojuegos, entre otras. Una vez que la aplicación web está en ejecución en los clientes web, el intercambio de información puede ocurrir con el pase de mensajes hacia *servicios web*. La lógica que se ejecuta en el cliente web y en los servicios al cual se comunica, puede ser desarrollados usando *modelos de programación* o *frameworks*, que pueden constar de un conjunto de herramientas que solucionan problemas comunes y esto ayuda en la *construcción de aplicaciones incrementales* y a la *actualización constante* de las aplicaciones. La información que es generada gracias a la

interacción de los *usuarios*, puede ser valiosa y es importante guardarla en sitios adecuados para ello, por lo tanto, es importante que se implemente un sistema de *gestión de base de datos* permitiéndole a los usuarios poder acceder en todo momento a una información íntegra y consistente.

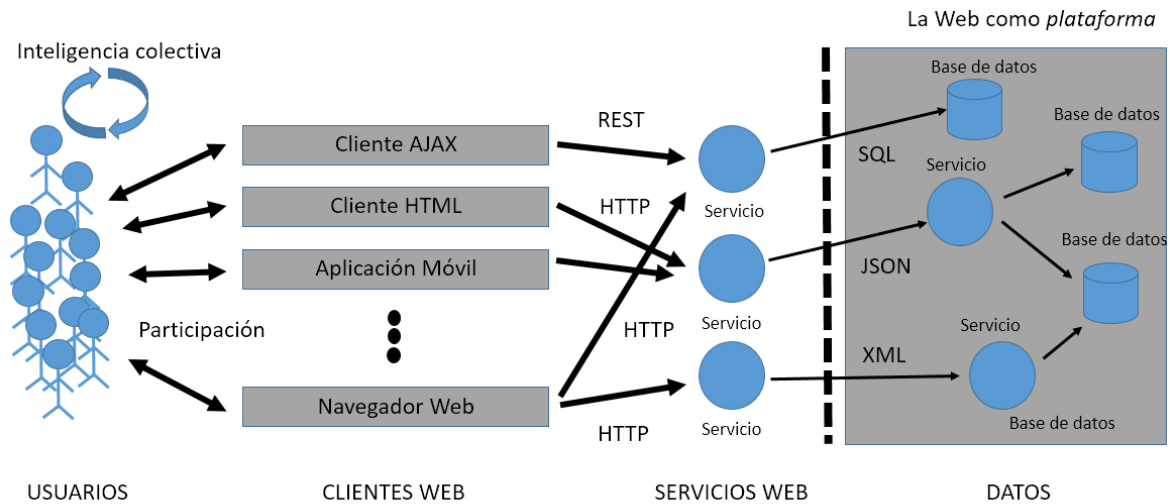


Figura 1.1: Interacción de los diferentes principios de la *Web 2.0*.

1.2. Aplicación web

Una aplicación web es cualquier programa que puede ser ejecutado en un navegador web. Este es creado usando lenguajes soportados por navegadores (como *HTML*, *Javascript* y *CSS*) para que luego pueda ser interpretados, mostrando el resultado al usuario. Las aplicaciones web son populares debido a lo práctico del navegador web como cliente ligero, a la independencia del sistema operativo, así como a la facilidad para actualizar y mantener aplicaciones web sin distribuir e instalar software a una gran cantidad de clientes. Existen aplicaciones como los emails web, wikis, portales web, tiendas en línea y la propia *Wikipedia* que son ejemplos bastante conocidos de aplicaciones web.

Antecedentes

En los primeros tiempos de la computación cliente-servidor, cada aplicación tenía su propio programa cliente que servía como interfaz de usuario que tenía que ser instalado por separado en cada ordenador personal de cada usuario. El cliente realizaba peticiones a otro programa (el servidor) que le daba respuesta. Una mejora en el servidor, como parte de la aplicación, requería normalmente una mejora de los clientes instalados en cada ordenador

personal, añadiendo un coste de soporte técnico y disminuyendo la productividad.

A diferencia de lo anterior, las aplicaciones web generan dinámicamente una serie de páginas en un formato estándar, como *HTML* o *XHTML*, soportados por los navegadores web comunes. Se utilizan lenguajes interpretados en el lado del cliente, directamente o a través de plugins escritos en Javascript, Java, Flash, entre otros, para añadir elementos dinámicos a la interfaz de usuario. Generalmente cada página web en particular se envía al cliente como un documento estático, pero la secuencia de páginas ofrece al usuario una experiencia interactiva. Durante la sesión, el navegador web interpreta y muestra en pantalla las páginas, actuando como cliente para cualquier aplicación web.

Interfaz

Las interfaces web tienen ciertas limitaciones en las funcionalidades que se ofrecen al usuario. Hay funcionalidades comunes en las aplicaciones de escritorio como dibujar en la pantalla o arrastrar/soltar que no están soportadas por las tecnologías web estándar. Los desarrolladores web generalmente utilizan lenguajes interpretados (scripts) en el lado del cliente para añadir más funcionalidades, especialmente para ofrecer una experiencia interactiva que no requiera recargar la página cada vez (lo que suele resultar molesto a los usuarios). Recientemente se han desarrollado tecnologías para coordinar estos lenguajes con las tecnologías en el lado del servidor. Como ejemplo, *AJAX* es una técnica de desarrollo web que usa una combinación de varias tecnologías, como también el protocolo *full duplex* que trabaja sobre *HTTP* llamado *WebSockets*.

Capítulo 2

Portal web

El término portal comúnmente es utilizado para referirse a alguna puerta o entrada a algún sitio. Hablando en el contexto de aplicaciones web, no difiere mucho, ya que un portal web es una puerta a diferentes sitios web, ya sea indexándolos a través de enlaces o creándolos en la misma aplicación.

Los portales web son a menudo la primera página que cargan los usuarios a conectarse en la web. Son muy utilizados para realizar páginas web que requieran modificar sus contenidos a menudo, ya que funcionan como un manejador de contenidos, haciendo que, cualquier modificación en la información se haga de manera rápida y sencilla.

El éxito de un portal depende de su habilidad de proveer un sitio base en donde los usuarios puedan regresar una vez finalizada la navegación a los demás sitios que lo componen.

2.1. Clasificación

En la actualidad no existe algún estándar para clasificar los portales web, recae directamente en el autor que los defina. De una manera muy general se pueden clasificar en dos (2) grandes grupos los cuales son:

2.1.1. Horizontal

Los portales horizontales se enfocan en obtener la atención de toda la comunidad. Estos sitios, son llamados comúnmente "megaportales", usualmente contienen motores de búsqueda y le proveen al usuario la habilidad de personalizar la página ofreciendo varios canales (como por ejemplo, información al clima regional, bolsa de valores, noticias).

2.1.2. Vertical

Los portales web verticales difieren solo en la información que quieren reflejar, la cual es mucho más específica sobre algún contexto. Son sitios que sirven como puerta a información relacionada a un negocio en particular, como pueden ser empresas, e-commerce, académicos entre muchos otros tópicos.

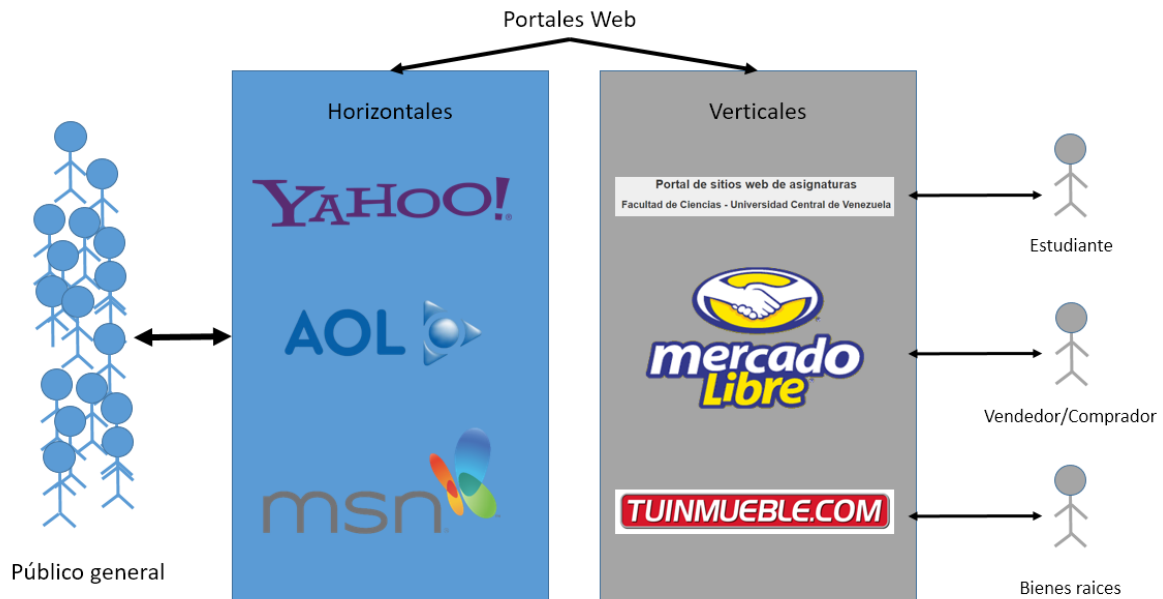


Figura 2.1: Diferencia entre portales *horizontales* y *verticales* en base a los usuarios que los utilizan.

En síntesis, la clasificación de los portales resalta el tipo de información que provee y la comunidad a la cual está dirigido. En la *Figura 2.1*, se puede observar que, los portales *horizontales* intentan llamar la atención de un *público general*, ya que la información que se encuentra dentro de estos es muy variada y no siguen un contexto específico. Un ejemplo clásico lo representa el portal web de *Yahoo!*.

Después, es apreciable, como los portales *verticales* buscan la atención de usuarios con necesidades e intenciones particulares. Un ejemplo muy cercano de este tipo de portales es el Portal de asignaturas de la Facultad de Ciencias de la UCV (*Portaliasig*) ya que la comunidad a la cual va orientada las funciones de dicho portal web es a una comunidad académica, específicamente, la comunidad académicas de la Facultad de Ciencias.

2.2. Elementos

Los portales web pueden ser muy diversos en cuanto al público al cual quieren obtener su atención o simplemente el contenido, pero todos los portales deben contener una serie de elementos en común. No todos los elementos a continuación deben estar presentes en cada uno de los portales existentes y algunos elementos pueden ser más importantes que otros dependiendo de la audiencia o la lógica del negocio de cada portal. Por ejemplo, un portal dirigido a usuarios anónimos, sin autenticación, podría ofrecer sin ningún problema un soporte limitado de transacciones seguras, integridad de los datos o alguna cualidad de colaboración entre usuarios. En cambio, un portal basado en el manejo de información y conocimientos puede enfocarse únicamente en motores de búsquedas e integración de los datos.

Los elementos claves de los portales web son los siguientes:

- Aspecto y apariencia.
- Seguridad.
- Perfil de usuario.
- Personalización.
- Taxonomía y navegación dinámica.
- Integración.
- Base de datos.
- Herramientas de colaboración.
- Motor de búsqueda.

Aspecto y apariencia

Como cualquier programa que interactúa con ser humano, los portales web tienen una apariencia o una interfaz gráfica de usuario (*GUI*). Muchos desarrolladores pueden no darle mucha importancia al aspecto o simplemente no preocuparse en lo absoluto, lo cual puede ser un error muy grave ya que, en el mundo de aplicaciones de software hechas para un usuario final, lo que realmente le importa al consumidor es que la aplicación se vea bien, sea usable e intuitiva, aparte de que logre las funcionalidades para las cuales fue creada. En el caso de los portales web, la página de inicio debe indicarle al usuario que está efectivamente , en un portal, ya sea por colocar en el título de *Portal* en el sitio o no, o si no, indicarles en todo momento en que sección o sitio web, alojado en el portal, están en un momento determinado y también, ofrecer una manera fácil e intuitiva de navegar a otra sección o simplemente regresar a la página de inicio.

Seguridad

Todo sistema en el cual se manejen cuentas de usuarios y el contenido que éstos pueden crear, colaborar o integrar, es importante establecer método de seguridad, ya sea, para poder identificar de manera unívoca a cada uno de los usuarios (autenticación) o evitar que un usuario particular pueda violentar los datos de otro usuario como también entrar a secciones prohibidas las cuales no tiene el privilegio (autorización y roles). El concepto primordial de un portal web es alojar múltiples sitios web en él, es importante que se maneje un sistema de autenticación que permita a los usuarios identificarse a lo largo de todo el contenido que el portal web provee.

Perfil de usuario

Como se mencionó anteriormente, es importante proveer la identificación única a cada usuario, pero no solo basta con eso. Es importante también que cada usuario tenga la posibilidad de poder ver o modificar en cualquier momento sus datos personales a través de un panel o sitio web que normalmente recibe el nombre de *perfil de usuario*.

Personalización

Un portal web debe ser capaz de proveer criterios donde los usuarios decidan como quieren que algún tipo de contenido le sea mostrado o la manera de los mensajes sean entregados a ellos. Es importante decir que cada usuario es único, y no solo eso, cada usuario, dependiendo del tipo de portal, puede pertenecer a una cantidad mayor de sitios web que otro usuario. Por ejemplo, en el caso del portal de asignaturas de la Facultad de Ciencias de la UCV (*Portalsig*), un estudiante puede pertenecer a una de las carreras que la facultad provee, por lo tanto es importante que, una vez que un estudiante es identificado en el sistema, el portal sea capaz de resaltarle la información que en verdad le puede interesar a un usuario y que, en este caso en particular, sería las materias que cursa actualmente.

Taxonomía

La taxonomía es una jerarquía de categorías usada para simplificar la navegación y la búsqueda. Las categorías están estrictamente relacionadas al tipo de contenido que provee un dicho portal. Por ejemplo, en *Portalsig*, siendo un portal de asignaturas de las carreras dictadas en la Facultad de Ciencias UCV, es importante tener una jerarquía Carrera/Materia/Sitios por semestre, ya que de esta manera se le da un correcto significado y navegación a las funcionalidades que puede realizar la comunidad de dicha facultad.

Integración

El límite de un portal web no debe ser su propio dominio de información. En el mundo de las aplicaciones de la *Web 2.0*, un concepto clave es la integración, que da vida a los llamados

servicios web que nacen con la necesidad de querer realizar una comunicación máquina-máquina.

Base de datos

Detrás de la mayoría de las aplicaciones web, como los portales web, reside una base de datos, usualmente relacional. En un portal web donde se requiere tener un sistema de autenticación, manejar roles, diferentes sitios web, es necesario una base de datos para poseer unos datos íntegros y disponibles en todo momento para el correcto funcionamiento del portal web.

Herramientas de colaboración

Los portales de hoy en día deberían contener herramientas que sirvan para la colaboración mutua entre usuario como, dar a conocer la presencia o no de un usuario, mensajería instantánea, comunidades virtuales, un sitio dedicado a discusiones y perfiles de usuario. Actualmente *Portalsig* posee la funcionalidad de que los usuarios, registrados a los sitios web de alguna materia, puedan participar en foros, que normalmente, son usados para dudas en proyectos o asignaturas.

Motores de búsqueda

Esta característica es sumamente importante, ya que recordando que un portal web es la *puerta* a diferentes sitios web, debe existir la posibilidad de que los usuarios puedan buscar algún contenido o material en alguno de estos sitios de manera rápida, sin tener que visitar cada uno de los sitios uno por uno hasta lograr encontrar lo deseado. Para esto nacen los motores de búsqueda que pueden ser tan simples como un campo de texto que tenga una lógica de indexación y una función de búsqueda en todos los sitios web pertenecientes.

Capítulo 3

Tendencias tecnológicas

En el mundo de las aplicaciones web y las herramientas que son utilizadas para construirlas, existen muchas actualizaciones y nuevas tecnologías que nacen cada día dejando a las más antiguas como obsoletas. El impulso de que nazcan nuevas tecnologías está relacionado mucho en como los clientes web, en los cuales las aplicaciones web se ejecutan, han evolucionado integrando funcionalidades cada vez más nuevas y nativas, como *HTML5*. Cada día estos clientes, sean navegadores o no, se vuelven más sofisticados, nacen nuevos estándares e Interfaces de Programación de Aplicaciones (*API*).

Una de las actualizaciones más importantes en el mundo web es la llegada de *HTML5*, que es la última versión del lenguaje más importante de la *WWW*, *HTML*, la cual trae muchas funcionalidades nuevas que, antes eran posibles con scripts escritos en *Javascript* como también la llamada *Web Semántica* la cual agrega nuevas las etiquetas o *tags* de *HTML* como por ejemplo, *header* y *footer* que su significado es para ser usados en la cabecera y el pie de una página web, respectivamente.

3.1. Arquitecturas y patrones

Las nuevas tendencias no se detienen a nivel de plataformas o *APIs* que la *Web* provee actualmente, sino también a nivel de patrones y arquitecturas. Uno de los patrones más usado a la hora de realizar una aplicación web es *MVC* (Modelo Vista Controlador) que básicamente su función es separar los datos de la lógica de negocio.

3.1.1. MVC

Como fue anteriormente mencionado, el patrón Modelo Vista Controlador (*MVC*) es uno de los patrones más usados a la hora de realizar una aplicación web. Como su nombre lo indica, consiste básicamente en tres (3) componentes y en como éstos interactúan uno con el otro para poder cumplir correctamente la lógica de negocio y las funcionalidades predefinidas de la aplicación.

Modelo

Es el ente encargado de representar todos los datos e información con la cual la aplicación opera, por lo tanto gestiona todos los accesos a dicha información, como las consultas o nuevas actualizaciones, implementando también los privilegios de accesos que estén previstos en una aplicación en específico (lógica de negocio).

La información que está preparada para ser mostrada (comúnmente a un usuario) es enviada a la *Vista*. Las peticiones de acceso o futura manipulación de los datos llegan al *Modelo* a través del *Controlador*.

Vista

Es la parte con la cual el usuario interactúa directamente. Se encarga de presentar el *Modelo* (información y lógica de negocio) en un formato adecuado para que un usuario pueda recibir la información correctamente.

La vista normalmente, es el resultado de renderización de archivos *HTML*, *CSS* y *Javascript*, pero también pueden ser simples lenguajes de comunicación como XML o JSON.

Controlador

Es el encargado de escuchar y responder los eventos (usualmente un usuario) e invoca peticiones al *Modelo* cuando se se hace alguna solicitud sobre la información, como editar algún documento o registro en una base de datos. También puede enviar cambios de presentación a la *Vista* asociada si se presenta un cambio en el *Modelo*, por lo tanto se puede considerar que el *Controlador* es un intermediario entre la *Vista* y el *Modelo*.

Al final, se posee un patrón de diseño donde la clave fundamental es la manera de cómo sus componentes colaboran entre sí y el usuario, como es posible observarlo en la *Figura 3.1*:

- El usuario a través de alguna acción, ya sea: ir a una ruta, mandar datos de un formulario, entre otras, *usa* algún procedimiento previsto en el *Controlador*.
- En el procedimiento ejecutado, según la lógica, puede o no haber alguna *manipulación* de los datos que dan vida a la lógica de negocio que se encuentran en el *Modelo*.
- El *Modelo* una vez que tenga los datos solicitados o simplemente haya terminado sus operaciones con los datos persistentes, es necesario dar una respuesta al usuario, por lo tanto se *actualiza* la *Vista*.
- Finalmente el usuario *observa* los cambios y notificaciones, para así poder realizar una futura acción en la aplicación web.

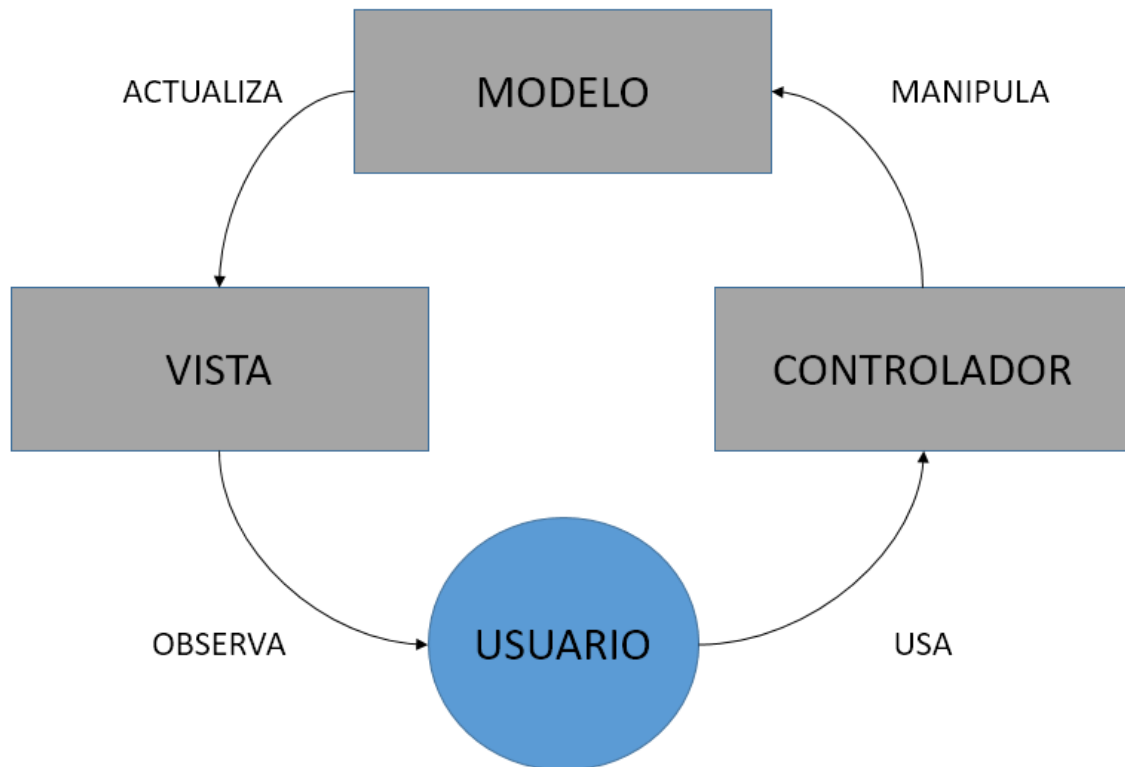


Figura 3.1: Colaboración de los componentes del patrón MVC.

3.1.2. ORM

El Mapeo Objeto-Relación (*Object-Relational mapping*, en su siglas en inglés) es una técnica de programación muy utilizada que sirve para conectar objetos propios de una aplicación, con tablas de un sistema manejador de base de datos. Usando un ORM, las propiedades y relaciones de los objetos de una aplicación puede ser fácilmente registrado y consultados de una base de datos sin tener que escribir sentencias *SQL* directamente, abstrayendo al desarrollador en su mayoría.

Esta técnica es comúnmente usada junto al patrón Modelo Vista Controlador (MVC), donde se obtiene que el *ORM* es parte fundamental del *Modelo*, ya que es el encargado de entablar una comunicación con un repositorio de datos, como por ejemplo, con un sistema manejador de base de datos, *Figura 3.2*.

Un ORM bien formado debería ser capaz de:

- Representar los modelos y sus datos.
- Representar asociaciones entre dichos modelos.

- Representar herencias jerárquicas a través de los modelos.
- Validar los modelos antes que estos sean persistidos en una base de datos.
- Permitir operaciones de base de datos de una manera orientada a objetos.

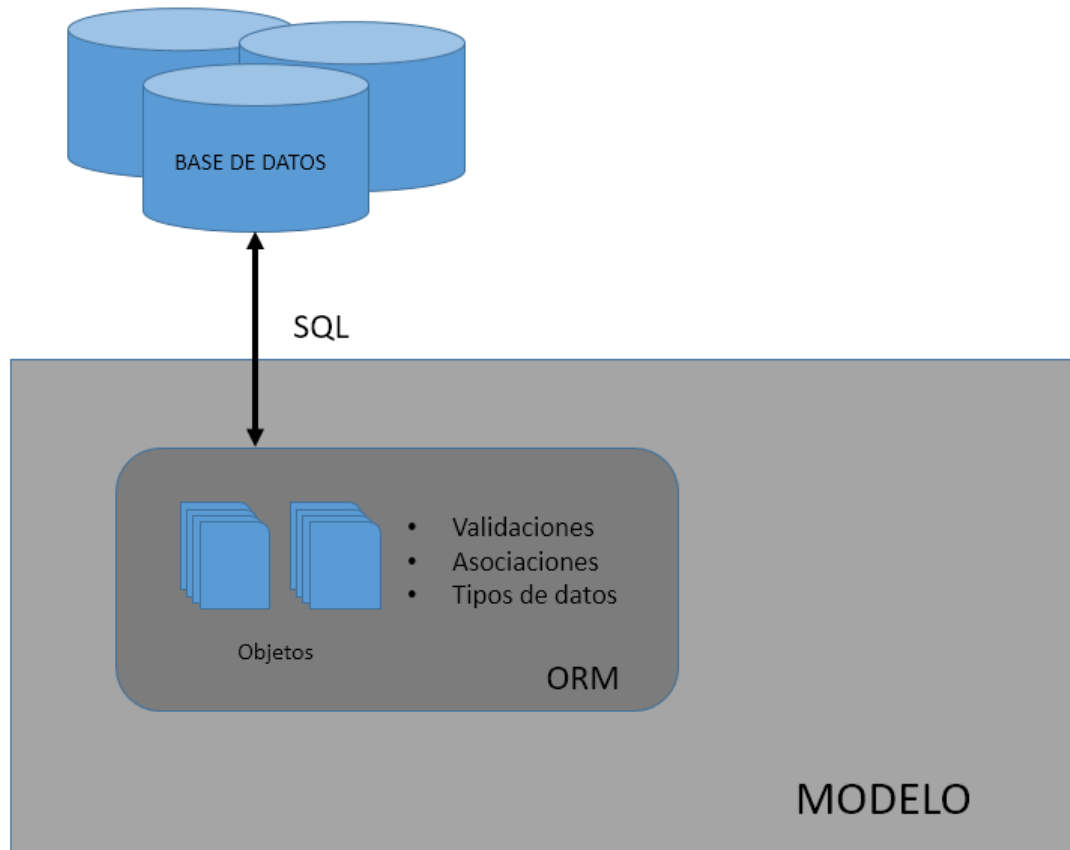


Figura 3.2: Integración de la técnica *ORM* con el *Modelo* del patrón *MVC*.

3.1.3. Web APIs y SPA

Unas de las tendencias con un auge considerado actualmente, es la realización de aplicaciones web usando una arquitectura que consta en dos (2) partes:

- La primera parte basada en recursos (*Rest*) del lado del servidor (o *Backend*).
- Y la segunda parte que se ejecutaría en el lado del cliente (*Frontend*), generalmente se usan páginas web enriquecidas o se usan un conjunto de técnicas para dar vida a una *Aplicación de una Sola página* (o *Single-page Application* en inglés).

Web APIs Rest

Rest es una arquitectura de desarrollo en *servicios web*, o mejor llamados *Web APIs*. El principio de Rest es utilizar el protocolo HTTP con “significado” en el sentido de:

- Usar correctamente los métodos HTTP; *GET* para consultas, *POST* para enviar datos, *PUT/PATCH* para modificar datos y *DELETE* para eliminar datos. Estas operaciones en conjunto llevan el nombre de *CRUD* (Crear, Leer, Actualizar, Eliminar, según sus siglas en inglés).
- Usar correctamente los códigos de estado de respuesta HTTP.
- Dar un buen uso a las cabeceras HTTP y a su significado. Una de las cabeceras más usadas es *Accept* la cual indica el formato de datos que espera o “acepta” el cliente que realice una petición.

SPA

Una Aplicación de una Sola Página (*Single-page application*, en inglés), es una aplicación web que encaja en una sola página web dando una experiencia al usuario similar a la de una aplicación de escritorio. Al igual que cualquier página web, es necesario código *HTML*, *Javascript* y *CSS*, solo que este es obtenido en su mayoría en la primera carga de una aplicación SPA. Esta primera carga usualmente recibe el nombre de *Bootstrapping*. Una vez que la primera carga es completada y el cliente web renderiza la página, no hay necesidad de recargar nuevamente y, los futuros cambios de contenidos se realizarán generalmente con *AJAX* o *WebSockets*.

Al final la interacción de un *Web API* con una aplicación en el lado del cliente SPA, sería de la siguiente manera, *Figura 3.3*:

- Se realiza la primera carga de archivos *HTML*, *CSS* y *Javascript*, que usualmente se le conoce como *Bootstrapping*.
- La lógica de la SPA estará en los archivos *Javascripts*, ya alojados en el cliente web.
- La actualización posterior de componentes de la aplicación web se realizarán mediante peticiones *HTTP* en segundo plano, que pueden ser realizadas con la técnica *AJAX*.
- El *Web API* al recibir peticiones *AJAX*, responderá con lenguajes de comunicación como *XML* o *JSON*.
- Una vez que el cliente web reciba las respuestas en *JSON* o *XML*, la lógica de la SPA se deberá de encargarse de actualizar de manera dinámica la *Vista* al usuario.

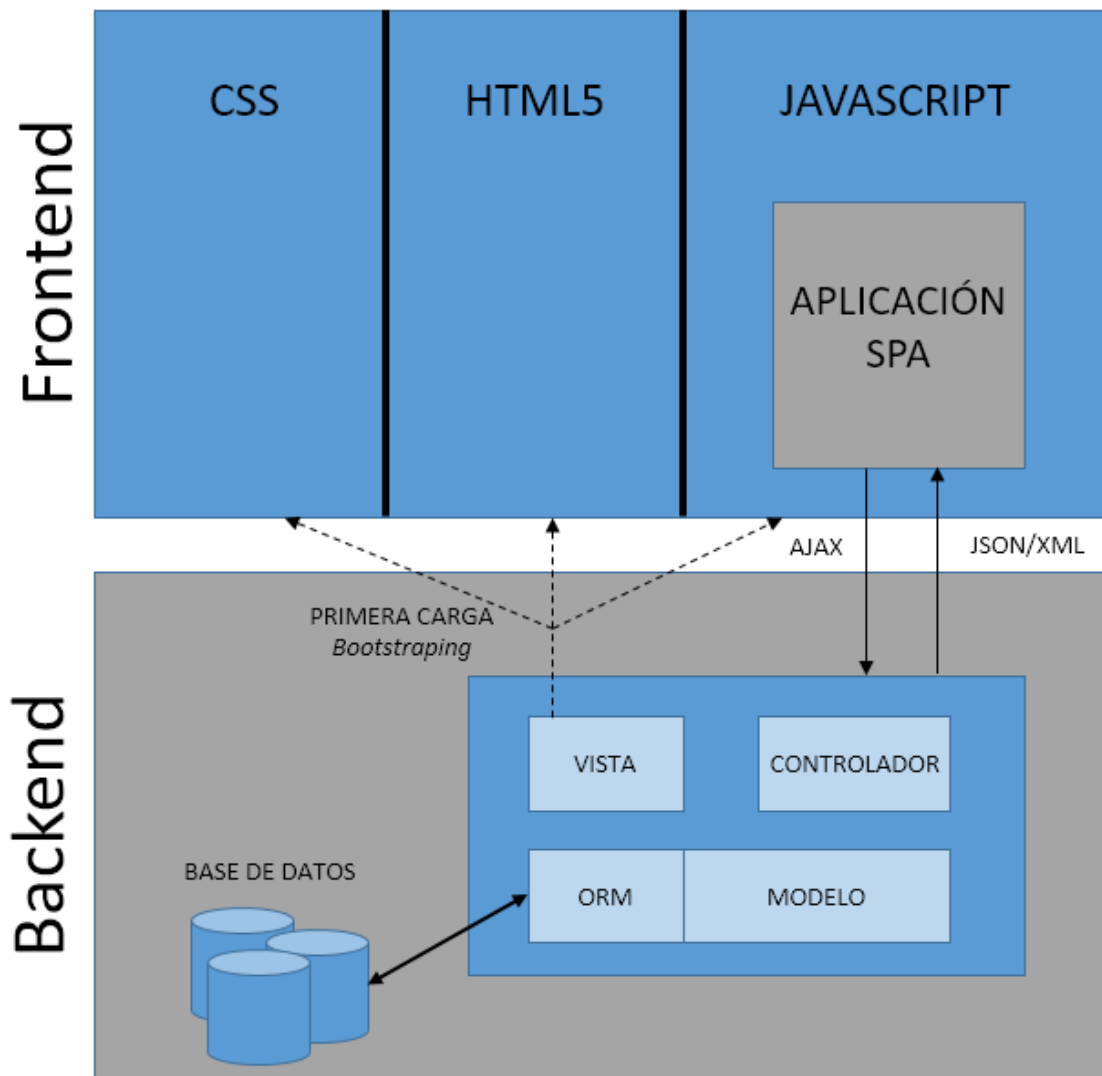


Figura 3.3: Arquitectura de una aplicación web que consta de un *Web API* y una *SPA*.

3.2. Lenguajes y Frameworks

Un lenguaje de programación es un vocabulario con un conjunto de reglas gramaticales que le indican a un computador realizar una tarea en específico. También podríamos definirlo como aquel lenguaje que permite especificar de manera precisa sobre que datos debe operar una computadora, como deben ser almacenados o transmitidos y que acciones debe tomar bajo una variada gama de circunstancias.

La descripción de un lenguaje de programación usualmente se divide en dos componentes: la sintaxis (la forma en que se escribe) y la semántica (lo que significa).

Suelen clasificarse entre interpretados y compilados, donde los interpretados tienen un intérprete específico que obtiene como entrada un programa y ejecuta las acciones escritas a medida que las va procesando, mientras que los compilados tienen un compilador específico que obtiene como entrada un programa y traduce las instrucciones las cuales pueden servir de entrada para otro intérprete o compilador.

Una de las ventajas a la hora de desarrollar una aplicación web es la gran cantidad de lenguajes que se pueden utilizar. Muchos de estos lenguajes son de propósito general, lo cual significa que están diseñados para escribir una gran cantidad de tipos de aplicaciones, que pueden ser aplicaciones de escritorio, sistemas manejadores de base de datos, cálculos matemáticos y estadísticos, diseño de imágenes, aplicaciones web y hasta un mismo sistema operativo.

Actualmente los lenguajes más populares para la realización de nuevas aplicaciones web recaen en el conjunto de los lenguajes que son interpretados y usualmente usados para crear scripts. Esto se debe a su simplicidad sintáctica, son débilmente tipados y orientado a objetos, lo cual hace que los desarrolladores no tengan que conocer muy bien el lenguaje y se encarguen únicamente de atacar la lógica de negocio de una aplicación en particular.

Es muy común que, en un universo de aplicaciones donde la lógica de negocio de cada una de ellas no tienen ninguna relación, se encuentren operaciones similares que se repiten una y otra vez. Por lo cual, nacen diversa cantidad de bibliotecas que solventan necesidades específicas y se encapsulan en un solo ambiente de desarrollo. Ese conjunto de bibliotecas, reciben el nombre de *Frameworks*.

3.2.1. Ruby

Ruby es un lenguaje de programación interpretado, reflexivo y orientado a objetos. Posee un gestor de paquetes y bibliotecas llamado RubyGems, donde cada uno de estos paquetes es denominado *Gema* (o *gem* en inglés).

Algunas de las características de este lenguaje son:

- Orientado a objetos.
- Cuatro (4) niveles de ámbito de variable: global, clase, instancia y local.
- Manejo de excepciones.
- Iteradores.
- Expresiones regulares nativas.
- Sobrecarga de operadores.

- Altamente portable.

Ruby on Rails

Ruby on Rails es un framework de desarrollo de aplicaciones web, código abierto escrito en el lenguaje de programación Ruby. Sigue el patrón Modelo Vista Controlador (*MVC*). Ruby on Rails se distribuye a través del manejador de paquetes RubyGems.

Es considerado un framework altamente escalable por la gran cantidad de soluciones que posee. Viene incluido con un ORM llamado ActiveRecord, que se basa en el patrón de diseño con el mismo nombre. Otro de sus módulos más importantes son: Action Mailer, que sirve para enviar correos electrónicos y Active Resource, el cual proporciona una infraestructura necesaria para crear de manera rápida y sencilla recursos *Rest*.

Sinatra.rb

Sinatra.rb es otro de los frameworks más usados en el lenguaje Ruby. Su diferencia más notable con respecto a Ruby on Rails, es que Sinatra no sigue el patrón *MVC*, en su lugar, Sinatra se enfoca de proveer una mínimas abstracción al desarrollador de las partes fundamentales de una comunicación *HTTP*, la *Petición* y la *Respuesta*, de esta manera se pueden crear aplicaciones web en Ruby con el mínimo esfuerzo.

3.2.2. Python

Python es un lenguaje de programación dinámico e interpretado.

- Su código fuente no declara los tipos de datos (variables, parámetros o métodos).
- Python obtiene el tipo de dato en tiempo de ejecución.
- Se elimina el tiempo de compilación.

Es un lenguaje multiparadigma, es decir, Python se adapta al programador:

- Paradigma funcional
- Paradigma imperativo.
- Paradigma orientado a objetos.

Django

Django es un framework de desarrollo web de código abierto, escrito en el lenguaje Python, que, al igual que Ruby on Rails, sigue el patrón de diseño *MVC*. Django pone énfasis en el reuso, la conectividad y extensibilidad de componentes, el desarrollo rápido y el principio *No te repitas* (DRY, del inglés *Don't Repeat Yourself*).

Flask

Flask es un framework web minimalista escrito en Python y basado en *Werkzeug*. Se puede relacionar perfectamente con Sinatra.rb ya que ambos son frameworks que no obligan al usuario a usar una herramienta o biblioteca particular, solo provee abstracciones usando objetos que representan la *Petición y Respuesta HTTP*.

3.2.3. Javascript

Javascript es un lenguaje de programación interpretado, basado en el estándar ECMAScript. Javascript se define como:

- Orientado a objetos.
- Basado en prototipos.
- Imperativo.
- Débilmente tipado.
- Dinámico.

Se utiliza principalmente en su forma del *lado del cliente*, implementado en la mayoría de los navegadores web modernos permitiendo mejoras en la interfaz de usuario y páginas web dinámicas, aunque existe también una forma de poder ejecutar Javascript del *lado del servidor*.

Node.js

Node.js es una plataforma construida sobre el motor V8 de Google que sirve para desarrollar aplicaciones de lado del servidor, aunque no se limita a eso.

Las aplicaciones en Node.js son escritas en el lenguaje de programación Javascript, y pueden ser ejecutadas sobre el ambiente de ejecución de Node.js en distintas plataformas como Mac OS X, Microsoft Windows, Linux y SunOS.

Node.js funciona como un contenedor del motor V8 de Google, que lo optimiza para trabajar mejor en contextos distintos a un navegador web, y además provee una serie de API's optimizados para ciertos casos de uso. A su vez, Node.js funciona en un ambiente totalmente asíncrono basado en eventos y no bloqueante, lo cual permite realizar muchas operaciones de entrada/salida con un costo mínimo de tiempo, comportamiento usual en aplicaciones web.

Es importante mencionar que Node.js no es un framework de desarrollo web. Node.js es una plataforma que permite la ejecución de código Javascript fuera de un navegador que está

optimizado para realizar y recibir peticiones *HTTP*.

Node.js incorpora varios “módulos básicos” compilados en el propio binario, como por ejemplo el módulo de red, que proporciona una capa para programación de red asíncrona y otros módulos fundamentales, como por ejemplo *Path*, *FileSystem*, *Buffer*, *Timers* y el de propósito más general *Stream*. Es posible utilizar módulos desarrollados por terceros, ya sea como archivos “.node” precompilados, o como archivos en Javascript plano. Los módulos Javascript se implementan siguiendo la especificación *CommonJS* para módulos, utilizando una variable de exportación para dar a estos scripts acceso a funciones y variables implementadas por los módulos.

Los módulos de terceros pueden extender Node.js o añadir un nivel de abstracción, implementando varias utilidades *middleware* para utilizar en aplicaciones web, como por ejemplo los frameworks Connect y Express. Pese a que los módulos pueden instalarse como archivos simples, normalmente se instalan utilizando el Node Package Manager (*npm*) que facilita al desarrollador, la compilación, instalación y actualización de módulos así como la gestión de las dependencias.

Express.js

Express.js es un framework de desarrollo para aplicaciones web usando Node.js. Está basado en un framework más viejo llamado Connect.js. Es importante destacar que Express.js más que un framework es un microframework muy ligero. “Micro” no significa que la aplicación web tiene que encajar en un único archivo de Javascript, aunque ciertamente puede. Tampoco significa que Express.js carece de funcionalidad. El “micro” en microframework significa que Express.js tiene como objetivo mantener el núcleo simple pero extensible. Este no toma muchas decisiones por el desarrollador, tales como la base de datos a utilizar. Esas decisiones que si hace por el desarrollador, son fáciles de editar. Todo lo demás depende de que necesite el desarrollador, de modo que Express.js puede ser todo lo que necesites y nada que no desees.

Sails.js

Sails.js es un framework web que trabaja sobre Node.js, más específicamente, sobre Express.js, usando las primitivas y abstracciones elementales que este provee.

Es un framework actualmente muy nuevo y en constante desarrollo. Unas de las características más fundamentales son que implementa el patrón *MVC* y viene incluido con un *ORM* llamado *Waterline*, el cual ha ganado su popularidad por la gran semejanza a ActiveRecord de Ruby on Rails.

Angular.js

Angular.js es un framework de desarrollo de aplicaciones web que usa tecnologías del lado del cliente (HTML, CSS, Javascript). Es un framework desarrollado y mantenido por Google. Es importante destacar de acuerdo a Brad Green y Shyam Seshadri (2013) que Angular.js se basa en el esquema de una aplicación de una sola página, donde básicamente nunca se hace una carga completa de la página sino de secciones específicas usando técnicas como AJAX. Para el desarrollo se basa en el patrón MV*, muy parecido a Modelo Vista Controlador, solo que en este caso el “Controlador” puede ser cualquier otro tipo de componente.

3.3. Base de datos

Una base de datos es un repositorio de información que contiene datos que se relacionan bajo alguna temática o lógica. Es una colección de datos organizados, que pueden ser esquemas, tablas, consultas, reportes, vistas y otro tipos de objetos. Toda base de datos necesita un sistema gestor que sirva de puente entre la base de datos en sí y algún programa de alto nivel o a un usuario que normalmente se le llama “administrador de base de datos”. Este sistema recibe el nombre de *Sistema Manejador de Base de Datos*. Los más conocidos son *MySQL*, *PostgreSQL*, *Oracle*, *MariaDB* y *MongoDB*.

3.4. Pilas de desarrollo web

Una pila de desarrollo es un conjunto de componentes de software que son necesarios para crear una plataforma completa. Luego se tiene que, las aplicaciones “corren en” o “corren sobre” dichas plataformas.

Por ejemplo, en el caso de una aplicación web, el desarrollador arquitecto del proyecto define una pila de desarrollo que puede consistir en:

- Sistema operativo.
- Servidor web.
- Base de datos.
- Lenguaje de programación.

3.4.1. LAMP

LAMP es una de las pilas de desarrollo más usadas en el mundo web a lo largo de los años. Su nombre es acrónimo de los nombres de sus componentes que son:

- Linux: El sistema operativo.

- Apache: El servidor web.
- MySQL: Sistema manejador de base de datos relacional.
- PHP: Lenguaje de programación.

Pueden existir ligeros variantes de esta pila, por ejemplo en el caso de *WAMP*, donde el sistema operativo es *Microsoft Windows*. También pueden haber cambios en el lenguaje de programación utilizado, como es el uso de *Perl* o *Python* y también cambios a nivel de base de datos, pueden ser usados *MariaDB* o *MongoDB*.

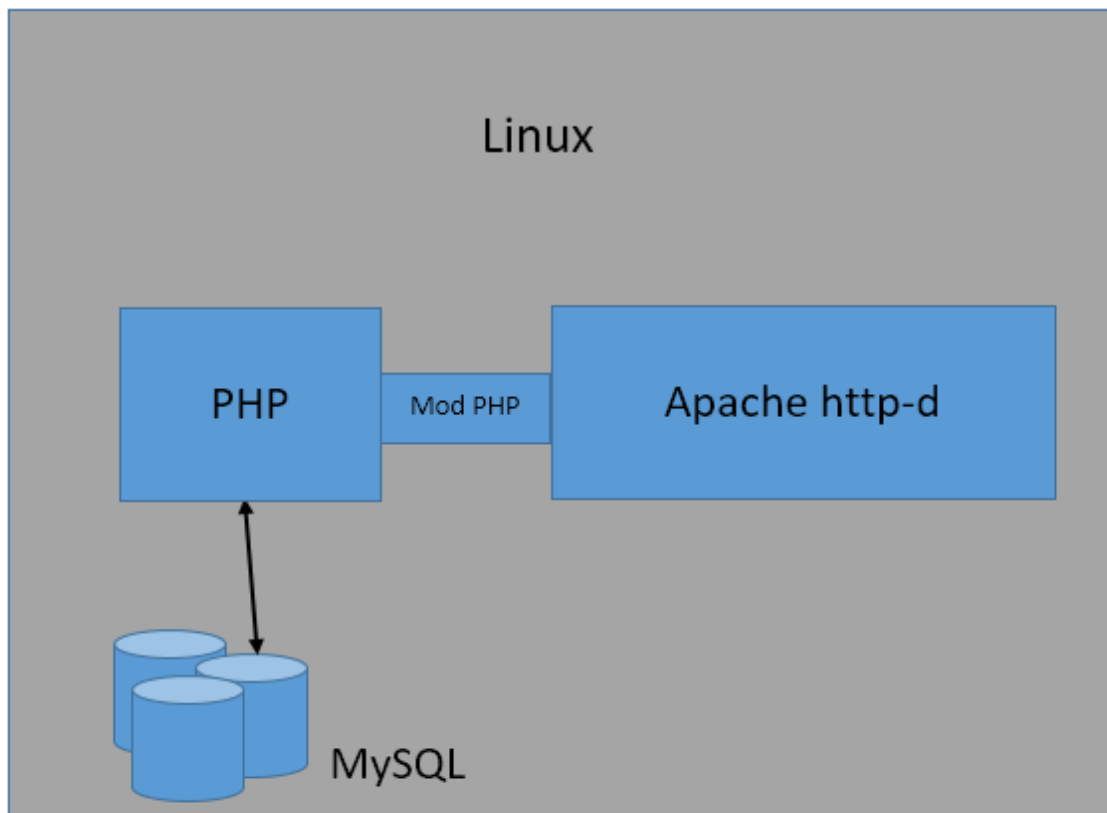


Figura 3.4: Pila de desarrollo web *LAMP*.

3.4.2. MEAN

MEAN es otra de las pilas de desarrollo web con mucho interés actualmente, es más novedosa que LAMP debido a el poco tiempo que tienen sus componentes en el mercado. Una de las características que más resalta de MEAN es que todos los componentes comparten el lenguaje de programación Javascript y el lenguaje de comunicación JSON (*Javascript Notation Object*).

Sus componentes son los siguientes:

- MongoDB: Una base de datos NoSQL.
- Express.js: Framework minimalista para aplicaciones web.
- Angular.js: Framework MV* para el desarrollo de SPAs.
- Node.js: Plataforma para realizar aplicaciones en lado del servidor.

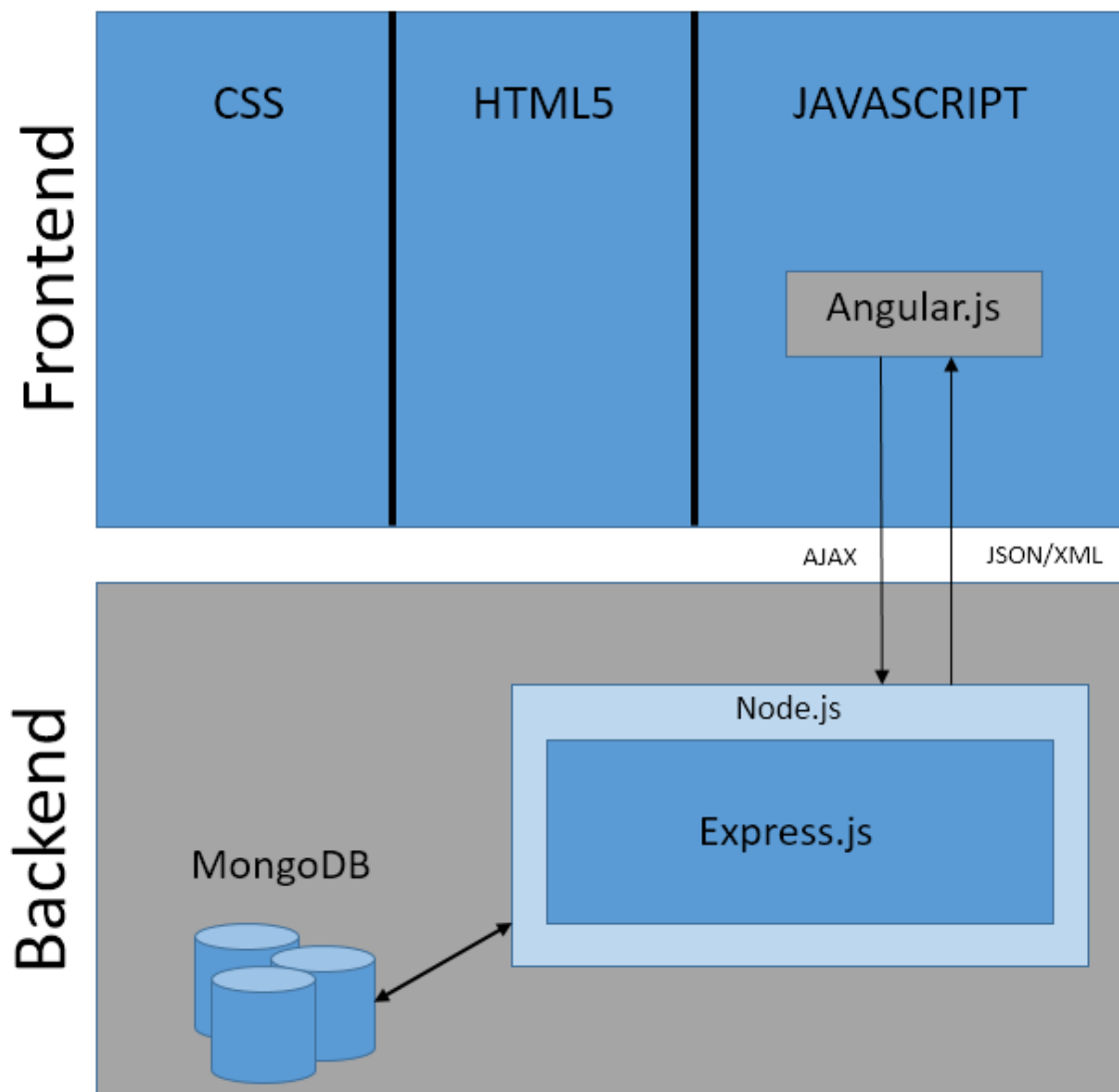


Figura 3.5: Pila de desarrollo web *MEAN*.

Capítulo 4

Portaliasig

El portal generador de sitios web de la Facultad de Ciencias de la Universidad Central de Venezuela (*Portaliasig*) es un portal web netamente vertical, ya que la comunidad la cual involucra no solo es de carácter académico sino también únicamente de la Facultad de Ciencias. Esta en producción desde el año 2013 en el Centro de Computación de dicha facultad. Este portal contiene entre sus funcionalidades, la creación de asignaturas, docentes, estudiantes y nuevos sitios web de asignaturas. Cada sitio generado por el portal, cuenta con funcionalidades para manejar el contenido de la asignatura clasificado en las siguientes secciones: información general, objetivos, bibliografía, contenido temático, grupo docente, horarios, estudiantes, evaluaciones, entregas, planificación, calificaciones, noticias, foros, descargas, enviar correo, semestres anteriores y registro de actividades.

La metodología de desarrollo utilizada para la realización de *Portaliasig* fue *AgilUs*, el cual es un método ágil de desarrollo por etapas que se enfoca en la usabilidad del producto final.

Como posibles modificaciones futuras, el equipo de desarrollo de *Portaliasig* concluyó:

- Un módulo para la conexión al sistema de gestión académica Conest, lo cual reduciría el trabajo del administrador para la carga de las asignaturas, docentes y estudiantes.
- Un servicio web que permita la carga de las calificaciones de manera automática desde el sitio web generado hacia el sistema de gestión académica *Conest*, lo que optimizaría la carga de notas.
- Una interfaz de programación de aplicaciones (API, por sus siglas en inglés) para teléfonos inteligentes, lo cual facilitaría el acceso al sitio web.
- Funcionalidades de accesibilidad que permitan la inclusión de los usuarios con discapacidades, como aumentar el tamaño de las fuentes, cambiar el contraste de los colores, entre otros.
- Un módulo que permita la personalización del diseño de los sitios web.

4.1. Funcionalidades generales

Entre las características y funcionalidades que posee actualmente *Portalsig* se separan por los roles de los usuarios, los cuales son: administrador, profesor, preparador, estudiante y público en general.

Administrador

- Agregar, modificar o eliminar asignaturas.
- Agregar, modificar o eliminar estudiantes.
- Agregar, modificar o eliminar profesores.

Profesor

- Crear un sitio web de asignatura nuevo.
- Agregar y eliminar a docentes, preparadores y auxiliares docentes que pertenezcan al grupo docente de un sitio web de una asignatura.

Profesor/Preparador

- Administrar información de la asignatura.
- Administrar secciones de estudiantes.
- Completar información pertinente a la asignatura, ya sea: información general, objetivos, bibliografía, contenido temático, grupo docente, horarios, estudiantes, evaluación, entregas, planificación, calificaciones, noticias, foros y descargas.
- Agregar evaluaciones que se realizan a lo largo del semestre.
- Agregar asignaciones a través de un modulo de entregas.
- Insertar los eventos que se vayan a realizar a lo largo del semestre en curso.
- Modificar calificaciones de distintas evaluaciones.
- Compartir contenido de noticias y foros a través de las redes sociales: Twitter, Facebook, Google+.
- Agregar archivos a la sección de descargas.
- Enviar un correo a todos los estudiantes cursantes de una asignatura.

Profesor/Preparador/Estudiantes

- Crear y participar en foros de una asignatura.
- Notificar vía correo electrónico cualquier cambio que se realice al sitio web en las secciones más relevantes, las cuales son: entregas, planificación, calificación, noticias, foros y descargas.

Todos los usuarios

- Permitir descargar de archivos agregados por el grupo docente.
- Visualizar la información de un sitio web creado.

4.2. Estado actual

Actualmente el portal generador de sitios web de la Facultad de Ciencias (*Portaliasig*) se encuentra en normal funcionamiento proveyendo las funcionalidades que fueron anteriormente descritas. Sin embargo, es pertinente que se mencione, que poseen una serie de limitaciones y dificultades en algunos aspectos, los cuales son:

- En el panel administrativo no se maneja paginación y a la hora de listar a todos los estudiantes la navegación es lenta y la interfaz se torna incómoda.
- La hora máxima de una entrega fue prevista que fueran las 11:59PM del día seleccionado. Actualmente eso no es cumplido, las descargas se cierran mucho antes, generando inconvenientes con los estudiantes a la hora de entregar sus asignaciones a través del portal.
- La funcionalidad de entregas consta de un sistema de subida de archivos para que posteriormente sean revisados por el grupo docente de una materia en particular. Actualmente, la subida de archivos con un peso mayor aproximado a 2MB no es posible ya que la petición es interrumpida después de unos segundos y el servidor retorna un código de error. Eso ocurre también con la funcionalidad de subir archivos para la sección de “Descargas” de un sitio web.
- El diseño actual del portal web no se adapta a dispositivos con pantallas de diferentes tamaños. Esto es un problema ya que sería deseable poder navegar con un diseño usable sin importar el dispositivo que se use.
- Si en un futuro se decide integrar *Portaliasig* con un servicio web, esto no será posible sin la modificación del código fuente del portal web, ya que actualmente no posee lógica de peticiones y respuestas usando lenguajes universales de comunicación como *JSON* o *XML*.

- El esquema de la base de datos presenta unos detalles de estándares. Existe tablas con nombres en inglés y también en español.

4.3. Tecnologías utilizadas

Para la realización del ambiente en producción de *Portalsig* se utilizaron las siguientes herramientas y frameworks:

- El lenguaje de programación Ruby, en su versión 1.9.3.
- El framework de desarrollo web Ruby on Rails, en su versión 3.2.13.
- El ORM incluido en Ruby on Rails, ActiveRecord.
- MySQL como sistema manejador de base de datos.
- Apache, el servidor web.
- Debian, sistema operativo.

Un esquema simple de la arquitectura actual de *Portalsig* seria el siguiente, *Figura 4.1*:

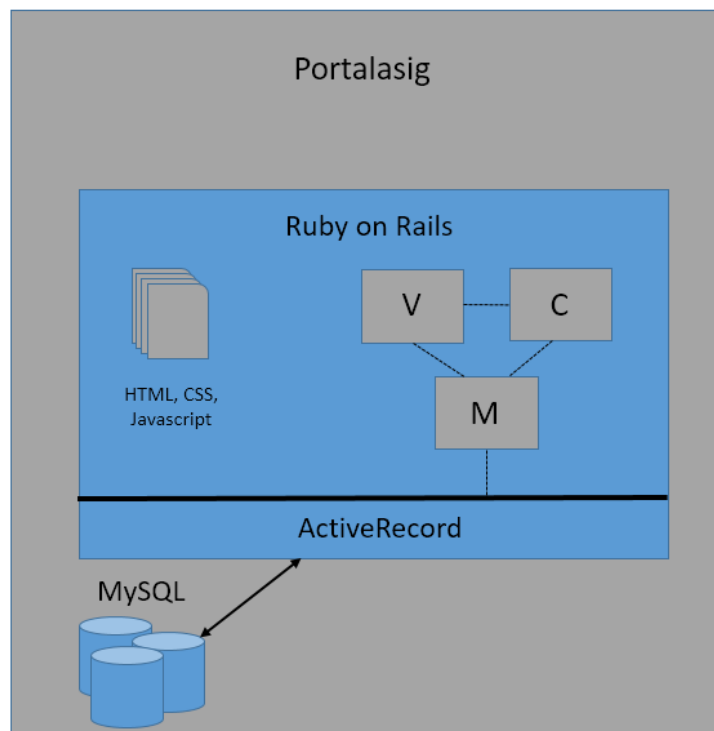


Figura 4.1: Arquitectura actual de *Portalsig*.

Capítulo 5

Propuesta de trabajo especial de grado

5.1. Planteamiento y justificación

Desde la llegada del Internet, las comunicaciones como se conocían hasta ese momento, cambiaron para realizarse de una forma que nunca antes había sido posible. Con la aparición de esta red de redes de comunicación descentralizada, que se comporta como una única red lógica de alcance global, se abren nuevas oportunidades de comunicación en todo el mundo, por lo que las personas otorgan cada vez más importancia al hecho de comunicarse y de compartir de una forma global sus conocimientos, opiniones, materiales, etc.

Haciendo uso del Internet, se han creado diversas soluciones de software que hacen el día a día de las personas que lo emplean más fácil y llevadero, como lo es el portal generador de asignaturas de la Facultad de Ciencias que se encuentra en producción actualmente (*Porta-lasig*), sin embargo, este se encuentra limitado en diversos ámbitos y posee algunos errores en algunas de sus funcionalidades. Por lo tanto, se busca la creación de un nuevo portal que conste de una interfaz de programación de aplicaciones y una aplicación web interactiva que se adapte a cualquier tipo de dispositivo, todo esto siguiendo un conjunto de estándares para obtener un código fuente limpio y legible, con su respectiva documentación. Es importante destacar que al poseer dicha interfaz se podrían elaborar las versiones móviles fácilmente.

5.2. Objetivo general

Desarrollar una nueva versión del portal generador de sitios web para la Facultad de Ciencias de la Universidad Central de Venezuela.

5.3. Objetivos específicos

- Mantener roles actuales: público, estudiante, preparador, profesor y administrador.

- Cambiar de una sola aplicación escalable a una aplicación de backend como web api (Rest) y una aplicación de frontend de una única página (SPA).
- Mantener los datos de los usuarios para que puedan acceder sin tener que ser creados nuevamente.
- Desarrollar un diseño responsive para que el portal web sea correctamente usable a la hora de ser accedido en dispositivos móviles (tablets o smartphones).
- Rediseñar el esquema de la base de datos.

5.4. Metodología

Se utilizara la metodología ágil *Scrum*. El cronograma constara de diez (10) semanas separadas por los siguientes *sprints*:

Número de sprint	Objetivos	Duración (semanas)
1	Migrar datos de usuarios	1
2	Realizar Web API	4
3	Documentar Web API	1
4	Realizar SPA	4

Tabla 5.1: Cronograma

5.5. Tecnologías

Para la realizacion de este futuro Trabajo Especial de Grado se planean utilizar la siguiente pila de desarrollo web, *Figura 5.1*:

- Node.js: La plataforma que permite la ejecución de Javascript fuera del navegador.
- Express.js: Módulo y framework que corre sobre Node.js. Provee soluciones y abstracciones con objetos basados en la Petición y Respuesta HTTP, como también la creación de rutas para la implementación de un Web API sencillo.
- Bookshelf.js: Módulo y ORM de Node.js. Se integra muy fácilmente a Express.js. Provee compatibilidad con sistemas manejadores de base de datos como MySQL y PostgreSQL.
- PostgreSQL: Sistema de manejador de base de datos relacional.
- Angular.js: Framework de desarrollo de *Frontend*, escrito en Javascript y es usado para crear aplicaciones de una sola página (SPA).

- Bootstrap: Framework de desarrollo de *Frontend*, es una combinación de archivos en Javascript y CSS, y es usado para realizar diseños y estructuras de una página web de manera rápida y sencilla.

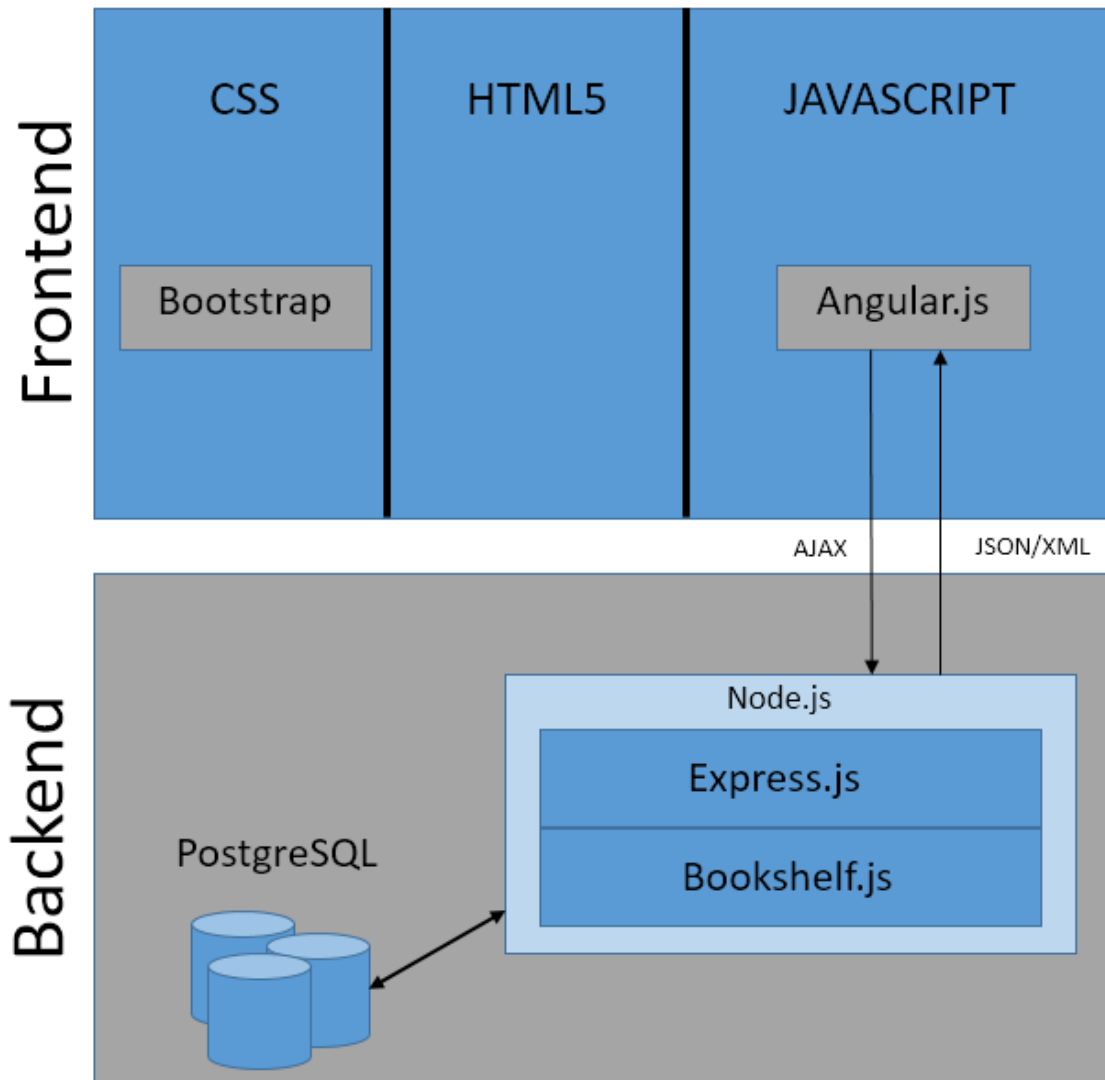


Figura 5.1: Arquitectura propuesta para la nueva versión de *Portalasig*.

5.6. Innovaciones propuestas

Entre las innovaciones más importantes propuestas para la nueva versión del portal de asignaturas tenemos:

- Cambio del esquema de la base de datos.

La nueva base de datos se basará en la ya existente, pero con algunos cambios relevantes, la ya existente posee una mezcla de inglés y español, esto se modificará para que se use como estándar solo el inglés, se agregarán tablas necesarias para poder manejar sesiones y administrar diversas funcionalidades como las preferencias del usuario, como también se eliminarán algunas relaciones cíclicas encontradas (por ejemplo entre la tabla evento y planificación) y la tabla mención, ya que es un portal de asignaturas para toda la facultad y hay carreras que no presentan esta característica.

- Cambio de la sección de carga usando HTML5 Multipart-Upload.

El sistema de carga de archivos actual presenta problemas a la hora de querer subir archivos con un peso aproximado mayor de 2MB, presentando una respuesta de error por parte del servidor. La solución propuesta, será utilizar la tecnología Multipart-upload, que consiste en separar un archivos en pequeños pedazos, de esta forma subir cada uno de los pedazos por separado. Se pretende usar la biblioteca Flow.js ajustada como módulo de Angular.js y de Node.js

- Diseño responsive.

El nuevo portal tendrá un diseño responsive, el cual se basa en que el sitio web se vea de manera adecuada en dispositivos con pantallas de diferentes tamaños, ya sea desktops, laptops, tablets o smartphones. Como paradigma de diseño se usará Mobile first (Móvil primero) que consiste en implementar las interfáces desde los dispositivos con pantallas más pequeñas primero para luego ir diseñando los dispositivos con el resto de dimensiones de pantallas.

- Panel de materias de interés al usuario.

A la hora de que un usuario se autentique de manera exitosa, la interfaz de la página principal cambiará ligeramente mostrando un panel donde se encuentren las materias asociadas a ese usuario en particular separándolas por semestres. En el caso de ser profesor, se mostraran las materias el cual el mismo dicta y en el caso de un estudiante, las materias el cual el está cursando como también materias el cual esté ejerciendo funciones como preparador, si es el caso.

- Preferencias del usuario.

El usuario ahora contará con preferencias personales con respecto al sistema de envío de correos, donde será posible seleccionar que servicios quiere que el sistema le notifique a través de un correo electrónico, como por ejemplo una nueva noticia, un foro nuevo creado, entre otras. Las notificaciones de modificación de una calificación para un estudiante no podrán ser deshabilitadas, ya que se considera muy importante que el estudiante se entere de manera correcta y rápida que un profesor o preparador modificó alguna de sus notas.

También habrán nuevos cambios en funcionalidades actuales, los cuales son:

- Paginación en el módulo de administración.
- Noticias usando el Web API de *Twitter*.
- Ingreso y modificación de notas con una apariencia parecida a *Microsoft Excel*.

Referencias

- [1] S. Ruby, *Agile Web Development with Rails 4*. The Pragmatic Programmers, 2013.
- [2] M. T. Gallego, *Metodología Scrum*.
- [3] M. Pilgrim, *Dive into Python 3*. Apress, 2011.
- [4] J. Townsend, Building Portals, Intranets, and Corporate Web Sites Using Microsoft Servers, 2004, https://books.google.co.ve/books?id=bCXnK31_d_0C&printsec=frontcover&dq=web+po#v=onepage&q&f=false.
- [5] A. Tatnall, Web Portals, the new gateways to internet information and services, 2005, [https://books.google.co.ve/books?id=5IeT8JhZSmcC&printsec=frontcover&dq=Tatnall+\(2005&hl=es-419&sa=X&ei=nfM0UrWgDZPU9ATn2YDACw&ved=0CDUQ6AEwAQ#v=onepage&q=Tatnall%20\(2005\)&f=false](https://books.google.co.ve/books?id=5IeT8JhZSmcC&printsec=frontcover&dq=Tatnall+(2005&hl=es-419&sa=X&ei=nfM0UrWgDZPU9ATn2YDACw&ved=0CDUQ6AEwAQ#v=onepage&q=Tatnall%20(2005)&f=false).
- [6] M. Haver, Eloquent Javascript, 2015, <http://eloquentjavascript.net/>.
- [7] T. O'Reilly, What Is Web 2.0? - Design Patterns and Business Models for the Next Generation of Software, 2005, http://www.im.ethz.ch/education/HS08/OReilly_What_is_Web2_0.pdf.
- [8] Top 10 web development trends and predictions for 2015, <http://www.zingdesign.com/top-10-web-development-trends-and-predictions-for-2015/>.
- [9] O. Díaz and M. Villavicencio, "Portal generador de sitios web de asignaturas para la facultad de ciencias de la universidad central de venezuela," Caracas, Octubre 2013.
- [10] S. Rivas, SPA Single Page Application, 2014, http://www.ciens.ucv.ve/portaliasig/aplicaciones_con_la_tecnologia_internet_ii/1-2014/descarga/descargar_archivo/1127.
- [11] —, Web Services, 2014, http://www.ciens.ucv.ve/portaliasig/aplicaciones_con_la_tecnologia_internet_ii/1-2014/descarga/descargar_archivo/982.
- [12] —, REST, 2014, http://www.ciens.ucv.ve/portaliasig/aplicaciones_con_la_tecnologia_internet_ii/1-2014/descarga/descargar_archivo/981.
- [13] R. Winkler, What's a Web Portal?, <http://www.atlanticwebfitters.ca/AboutCMS/WhatisaWebPortal/tabid/95/Default.aspx>.